

MEMBER 2

Spark Analytics & ETL

CSV → Parquet · 7 aggregate tables · CSV exports · ClickHouse loads

Name: _____

The shared table contract, metric definitions, risks and timeline are in **00_Team_Overview**. Read it first. This brief only covers your role.

YOUR FIRST 3 HOURS

- Hour 1: confirm the 8-table contract and the metric definitions with the team.** You compute every number, so settle any doubt now.
- Schema audit of `data/raw/`.** Check the real CSV columns. BTS rows end with a trailing comma, so select columns by name.
- Draft the explicit column list for CSV → Parquet** (the `COLS` dictionary in the starter below). Keep ~35 of the 109 columns.
- Work on the 1-month sample first: `2024_1.csv`** (~550k rows, about 10 s per run).

Your deliverables

- `spark_csv_to_parquet.py`
- `spark_aggregates.py` (Tables 1–7)
- Sample CSVs for Member 4 (hour 8 and hour 12)
- Aggregates loaded into ClickHouse (sample first, final data later)
- Verification queries for Q1–Q8
- Code cleanup and docstrings
- Content for slides 6, 7 and 8

Definition of done

- ☐ Contract and metric definitions confirmed at hour 1.
- ☐ `spark_csv_to_parquet.py` reads with an explicit column list (no `inferSchema`), casts types, derives `Dep_Hour`, writes Parquet partitioned by `Year` to `/project/flights/parquet/`.
- ☐ `spark_aggregates.py` builds Tables 1–7 from Parquet only. Column names match the DDL exactly.
- ☐ CSVs of Tables 1 and 4 sent to Member 4 by **hour 8**.
- ☐ All sample CSVs sent to Member 4 by **hour 12**.
- ☐ Sample aggregates loaded into ClickHouse and validated against raw numbers.
- ☐ CSV → Parquet run **once** on all 48 months (hours 24–30).
- ☐ Final aggregates bulk-loaded into ClickHouse. Row counts verified.
- ☐ Verification queries for Q1–Q8 run.
- ☐ Code cleaned up with docstrings.
- ☐ Content ready for slides 6, 7, 8.

Your schedule, hour by hour

Hours count from kickoff. Text is taken from the plan's schedule tables.

When	What you do (from the plan)
Day 1 — foundation, parallel work on the 1-month sample (hours 0–24)	
Hours 0–3	Schema audit of <code>data/raw/</code> . Confirm the 8-table contract + metric definitions with the team (hour 1) . Draft the explicit column list for CSV→Parquet.
Hours 3–8	Run CSV→Parquet on the sample. ETL core: casts, <code>Dep_Hour</code> , null handling. Produce Tables 1 and 4 and send CSVs to Member 4 by hour 8 .
Hours 8–14	Tables 2, 3, 5, 6, 7 on the sample. Send all sample CSVs by hour 12 .

When	What you do (from the plan)
Hours 14–20	Load the sample aggregates into ClickHouse via CSV (see the CSV load method below).
Hours 20–24	Validate all ClickHouse tables against raw numbers.
Day 2 — full 48-month run, polish, defense readiness (hours 24–48)	
Hours 24–30	Run CSV→Parquet once on all 48 months. Then run aggregates from Parquet.
Hours 30–36	Bulk-load final aggregates into ClickHouse; verify row counts.
Hours 36–42	Run verification queries for Q1–Q8 for the report.
Hours 42–48	Code cleanup and docstrings.

Handoffs: who sends what, by when

You receive from	What	By when
All members	The agreed contract: 8 tables + metric definitions	Hour 1
Member 1	1-month sample in HDFS: <code>/project/flights/sample</code>	Hours 3–8 †
Member 1	The ClickHouse tables, created from the DDL, before you load	Hours 14–20 †
Member 1	48 CSVs in <code>/project/flights/raw/</code>	Hours 24–30 †

You send to	What	By when
Member 4	Sample CSVs of Tables 1 and 4	Hour 8
Member 4	All sample CSVs	Hour 12
Member 4	Sample aggregates loaded in ClickHouse (Member 4 tests in hours 20–24)	Hours 14–20
Member 1	<code>spark_csv_to_parquet.py</code> and <code>spark_aggregates.py</code> , for the Airflow DAG	Hours 20–24 †
Member 4	Final aggregates loaded into ClickHouse, row counts verified	Hours 30–36
Team	Q1–Q8 verification queries, for the report	Hours 36–42
Member 4	Content for slides 6–8	Before slides 1–6 (hours 24–30) and 7–12 (hours 36–42) are built †

† Not written as a handoff in the plan, but the schedule needs it. Agree the exact hour with the other member.

The 7 tables you produce

#	Table	Answers
1	<code>agg_airline_performance</code>	Q1, Q6 (rates), Q8 (sum <code>Total_Flights</code> by <code>Year</code>)
2	<code>agg_airport_performance</code>	Q2, Q6 (rates)
3	<code>agg_delay_causes_monthly</code>	Q4
4	<code>agg_hourly_delays</code>	Q3
5	<code>agg_route_traffic</code>	Q7
6	<code>agg_calendar_delays</code>	Q5, Q8
7	<code>agg_cancellation_reasons</code>	Q6 (reasons)

Column names and types are in **00_Team_Overview** (table contract). Names must match the DDL. Order does not matter.

Metric definitions you implement

Copy of the shared definitions. 00_Team_Overview is the master.

- **Total_Flights** = all rows, including cancelled and diverted.
- **Operated flight** = `Cancelled = 0 AND Diverted = 0 AND ArrDel15 not null`.
- **Delayed_Flights** = operated with `ArrDel15 = 1`; **OnTime_Flights** = operated with `ArrDel15 = 0`.
- **Delay_Rate_Pct** = `Delayed / (Delayed + OnTime) × 100`. **Cancellation_Rate_Pct** = `Cancelled / Total_Flights × 100`.
- **Dep_Delay_Rate_Pct / Arr_Delay_Rate_Pct** = share with `DepDel15 = 1 / ArrDel15 = 1`.
- **Avg_*_Delay_Min** = average of `DepDelayMinutes / ArrDelayMinutes` (early = 0) over operated flights.
- **Total_Delay_Minutes** = sum of the five cause columns (null → 0). **Cause_Pct** = `cause minutes / Total_Delay_Minutes × 100`.
- **Dep_Hour** = `floor(CRSDepTime / 100) % 24`. **Day_Of_Week**: 1 = Monday ... 7 = Sunday (BTS `DayOfWeek`).
- **Season** (Power BI): Dec–Feb Winter, Mar–May Spring, Jun–Aug Summer, Sep–Nov Fall.
- **Route** is directed (`Origin-Dest`). **Cancellation_Reason**: A = Carrier, B = Weather, C = NAS, D = Security.

Data quirks to handle in the ETL

- BTS CSV rows end with a trailing comma, which creates an extra empty column. Select columns **by name** (do not rely on position).
- `CRSDepTime` is in **hhmm** format (e.g., 1530) and can be 2400. Use `Dep_Hour = floor(CRSDepTime / 100) % 24`.
- Delay-cause columns are only populated for flights with `ArrDel15 = 1`; otherwise null. Treat null as 0 when summing.
- `DepDelay/ArrDelay` are null for cancelled flights, and `ArrDelay` is null for diverted flights.
- Numeric fields come as strings like "-3.00". Cast explicitly.

Warnings for your role

- **Parquet, not raw CSV.** One job reads the 48 CSVs with an explicit column list (no `inferSchema`), keeps ~35 of 109 columns, casts types, derives `Dep_Hour`, and writes Parquet partitioned by `Year` to `/project/flights/parquet/`. Aggregations and ML read only this Parquet. Raw CSV runs take about 30 minutes. Parquet runs take a few minutes (~1.5–2 GB).
- **Run CSV → Parquet once on all 48 months** (hours 24–30). Every later run reads Parquet.
- **Use the metric definitions exactly** so every member gets identical numbers. `Total_Flights` counts all rows, including cancelled and diverted.
- **Column names must match the DDL.** Order does not matter.
- **No JDBC to ClickHouse.** It often fails on type mismatches (`UInt32` vs. Spark signed types) and needs extra jars. Use the small CSV loads below.
- **The code below is a starter, not tested against your cluster.** Adjust paths and options. Run it on the 1-month sample first.
- **Never cut:** CSV → Parquet and the 7 aggregate tables.

Commands, DDL and code

CSV → Parquet: run once (Python / PySpark starter)

```
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.appName("flights_csv_to_parquet").getOrCreate()

RAW = "hdfs:///project/flights/raw/*.csv"          # sample: .../sample/2024_1.csv
OUT = "hdfs:///project/flights/parquet"

COLS = { # column -> type (selected by NAME; BTS rows end with a trailing comma)
    "Year": "int", "Month": "int", "DayOfMonth": "int", "DayOfWeek": "int", "FlightDate": "date",
    "Reporting_Airline": "string", "Tail_Number": "string",
    "Flight_Number_Reporting_Airline": "string",
    "Origin": "string", "OriginCityName": "string", "OriginState": "string",
    "Dest": "string", "DestCityName": "string", "DestState": "string",
    "CRSDepTime": "double", "DepTime": "double", "DepDelay": "double",
    "DepDelayMinutes": "double", "DepDel15": "double",
    "CRSArrTime": "double", "ArrTime": "double", "ArrDelay": "double",
    "ArrDelayMinutes": "double", "ArrDel15": "double",
    "Cancelled": "double", "CancellationCode": "string", "Diverted": "double",
    "CRSElapsedTime": "double", "ActualElapsedTime": "double", "AirTime": "double",
    "Distance": "double", "CarrierDelay": "double", "WeatherDelay": "double",
    "NASDelay": "double", "SecurityDelay": "double", "LateAircraftDelay": "double",
}

df = spark.read.option("header", True).csv(RAW)          # all strings: no inferSchema pass
df = df.select([F.col(c).cast(t).alias(c) for c, t in COLS.items()])
df = (df.withColumn("Dep_Hour", (F.floor(F.col("CRSDepTime") / 100) % 24).cast("int"))
      .withColumn("Route", F.concat_ws("-", "Origin", "Dest")))

df.write.mode("overwrite").partitionBy("Year").parquet(OUT)
```

Aggregate → CSV (Python; repeat per table)

```
import os
os.makedirs("/tmp/out", exist_ok=True)
t1.toPandas().fillna(0).to_csv("/tmp/out/agg_airline_performance.csv", index=False) # repeat per table
```

CSV → ClickHouse (bash)

```
clickhouse-client --query "INSERT INTO flight_analytics.agg_airline_performance FORMAT CSVWithNames" \
  < /tmp/out/agg_airline_performance.csv
# Docker: docker exec -i clickhouse clickhouse-client --query "..." < file.csv
```